プログラミングワークショップ２

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

今後の予定

回	内容
~~1~~	~~ソフトウェア開発1~~
~~2~~	~~ソフトウェア開発2~~
~~3~~	~~ソフトウェア開発3~~
~~4~~	~~ソフトウェア開発4~~
~~5~~	~~中間発表1(テーマ発表)~~
~~6~~	~~ソフトウェア開発5~~
~~7~~	~~ソフトウェア開発6~~
~~8~~	~~ソフトウェア開発7~~

回	内容
~~9~~	~~ソフトウェア開発8~~
~~10~~	~~中間発表2(開発状況)~~
~~11~~	~~中間発表2(開発状況)つづき~~
12	ソフトウェア開発9
13	ソフトウェア開発10
14	ソフトウェア開発11(補講)
15	最終発表(補講)

- ソフトウェア開発：12回
- 中間発表：2回（テーマ発表、開発状況）
- 最終発表：1回

具体的な操作例から考える

- 例1)123+45=
  どのキーを押したとき何が表示されるか

押したキー	表示	
-	0	→ 初期状態
1	1	1項目の数値を入力中
2	12	
3	123	
+	123	→ 1項目の数値を入力完了、計算方法決定
4	4	2項目の数値を入力中
5	45	
=	168	→ 2項目の数値を入力完了、計算実行
C	0	→ 消去、初期状態に戻る

必要な変数・フィールドを考える

- フィールド(候補)
  - 1項目の数値
  - 2項目の数値
  - 入力途中の数値を保存する変数
  - 計算方法を保存する変数
  - 計算結果

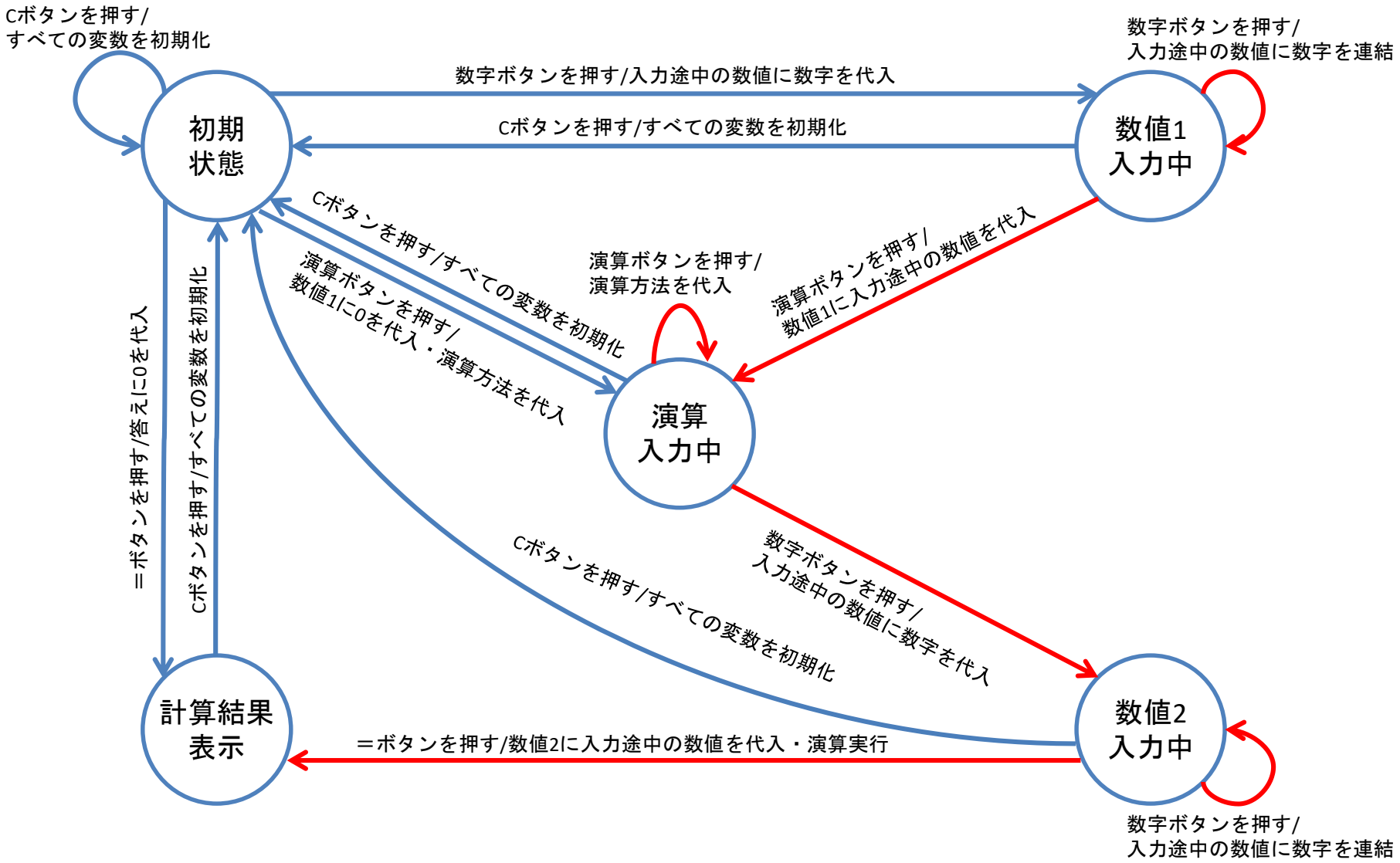
どんな状態があるか

- 状態の候補
  - 初期状態
  - 数値1入力中
  - 演算入力中
  - 数値2入力中
  - 計算結果表示
- 各々の状態で何のボタンを押したら次の状態に移るか考えよう

状態遷移表

	初期状態	数値1 入力中	演算 入力中	数値2 入力中	結果 表示中
数字ボタンを押す	入力途中の数値に数字を代入	入力途中の数値に数字を連結	入力途中の数値に数字を代入	入力途中の数値に数字を連結	
	数値1入力中	数値1入力中	数値2入力中	数値2入力中	
演算ボタンを押す	数値1に0を代入 演算方法を代入	数値1に 入力途中の数値を代入	演算方法を代入		
	演算入力中	演算入力中	演算入力中		
＝ボタンを押す	答えに0を代入			数値2に入力途中の数値を代入・演算実行	
	結果表示中			結果表示中	
Cボタンを押す	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化	すべての変数を初期化
	初期状態	初期状態	初期状態	初期状態	初期状態

状態遷移図


Cボタンを押す/
すべての変数を初期化
初期
状態
数字ボタンを押す/入力途中の数値に数字を代入
Cボタンを押す/すべての変数を初期化
数値1
入力中
数字ボタンを押す/
入力途中の数値に数字を連結
Cボタンを押す/すべての変数を初期化
演算ボタンを押す/
演算方法を代入
演算ボタンを押す/
数値1に入力途中の数値を代入
演算ボタンを押す/
数値1に0を代入・演算方法を代入
演算
入力中
＝ボタンを押す/答えに0を代入
Cボタンを押す/すべての変数を初期化
Cボタンを押す/すべての変数を初期化
数字ボタンを押す/
入力途中の数値に数字を代入
計算結果
表示
＝ボタンを押す/数値2に入力途中の数値を代入・演算実行
数値2
入力中
数字ボタンを押す/
入力途中の数値に数字を連結

列挙型

- 電卓の状態を表す型(State)を列挙型で表現
- 列挙定数
  - 初期状態　INIT
  - 数値1入力中　INPUT_NUM1
  - 演算入力中　SELECT_OP
  - 数値2入力中　INPUT_NUM2
  - 結果表示中　RESULT
- Calcクラスに電卓の状態変数stateを用意

```
//電卓クラス

enum State {
  INIT,           //初期状態
  INPUT_NUM1,     //数値1入力中
  SELECT_OP,      //演算入力中
  INPUT_NUM2,     //数値2入力中
  RESULT          //結果表示中
};

class Calc {

  State state;    //電卓の状態



}
```

Switch文を使った状態遷移の実装

- 状態変数stateで分岐
- 各状態で処理を記述
  - 必ず実行する処理
  - イベント発生時の処理

```
//電卓クラス
class Calc {

  void run(){
    switch(state){
    case INIT:
    // 初期状態の処理をここに実装
      break;
    case INPUT_NUM1:
    // 数値1入力中の処理をここに実装
      break;
    case SELECT_OP:
    // 演算入力中の処理をここに実装
      break;
    case INPUT_NUM2:
    // 数値2入力中の処理をここに実装
      break;
    case RESULT:
    // 結果表示中の処理をここに実装
      brek;
    default:
      break;
    }
  }
}
```

初期状態①

- 状態遷移図・表を参考に初期状態の処理を実装
- 必ず実行すること
  - 画面を表示する
  - 各ボタンを表示する
  - 入力途中の数値を表示

```
case INIT:
   //必ず実行する処理
   display(input);

   //各イベントの処理
   if(数字ボタンを押す) {
     // 入力途中の数値に押した数字を代入
     // 数値1入力中に遷移

   } else if(演算ボタンを押す){
     //数値1に0を代入 演算方法を代入
     // 演算入力中に遷移

   } else if(＝ボタンを押す){
     //答えに0を代入
     //結果表示中に遷移

   } else if(Cボタンを押す){
     //すべての変数を初期化
     //初期状態に遷移

   }
   break;
```

初期状態②

- イベント発生時の処理
  - 数字ボタンを押した場合
  - 演算ボタンを押した場合
  - ＝ボタンを押した場合
  - Cボタンを押した場合

```
case INIT:
   //必ず実行する処理
   display(tempNum);

   //各イベントの処理
   if(数字ボタンを押す) {
     // 入力途中の数値に押した数字を代入
     // 数値1入力中に遷移

   } else if(演算ボタンを押す){
     //数値1に0を代入 演算方法を代入
     // 演算入力中に遷移

   } else if(＝ボタンを押す){
     //答えに0を代入
     //結果表示中に遷移

   } else if(Cボタンを押す){
     //すべての変数を初期化
     //初期状態に遷移

   }
   break;
```

メインタブ

```
//メインタブ sample03

Calc calc;

void setup() {
  size(400, 475);
  calc = new Calc();
}

void draw() {

  calc.run();

}
```

Buttonクラス

```
//Buttonクラス
class Button {

  int x, y, w, h;
  String name;

  int buttonColor = color(255);
  int hoverColor  = color(0, 255, 255);
  int pushColor   = color(255, 255, 0);
  int textColor   = color(0);

  Button(int _x, int _y, int _w, int _h, String
_name){
    x = _x;
    y = _y;
    w = _w;
    h = _h;
    name = _name;
  }

  String getName(){}                //追加
  boolean isClicked(){}             //追加

  boolean isHover(){
    if((mouseX > x) && (mouseX < x+w) &&
       (mouseY > y) && (mouseY < y+h)){
      return true;
    } else {
      return false;
    }
  }
  boolean isPressed(){
    if(isHover() && mousePressed){
      return true;
    } else {
      return false;
    }
  }
  void display(){
    if(isPressed()){
      fill(pushColor);
    } else if(isHover()){
      fill(hoverColor);
    } else {
      fill(buttonColor);
    }
    rect(x, y, w, h);

    textAlign(CENTER, CENTER);
    textSize(32);
    fill(textColor);
    text(name, x, y, w, h);
  }
}
```

Screenクラス

```
//Screenクラス
class Screen {

  int x, y, w, h;

  int screenColor = color(0);
  int textColor   = color(255);

  Screen(int _x, int _y, int _w, int _h){
    x = _x;
    y = _y;
    w = _w;
    h = _h;
  }

  void display(String s){
    fill(screenColor);
    rect(x, y, w, h);

    textAlign(RIGHT, CENTER);
    textSize(48);
    fill(textColor);
    text(s, x+10, y+10, w-20, h-20);
  }

}
```

Calcクラス

```
enum State {
  INIT,
  INPUT_NUM1,
  SELECT_OP,
  INPUT_NUM2,
  RESULT
};

class Calc {

  Screen screen;
  Button[] btn;

  float num1;      //数値1
  float num2;      //数値2
  String input;    //入力途中の数値
  String ope;      //演算方法
  float answer;    //答え
  State state;     //電卓の状態
```

```
Calc() {
   num1 = 0.0;
   num2 = 0.0;
   answer = 0.0;
   input = "0";
   ope = "";
   state = State.INIT;

   screen = new Screen(0, 0, 400, 100);
   btn = new Button[17];
   btn[0]  = new Button(0, 400, 200, 75, "0");
   btn[1]  = new Button(0, 325, 100, 75, "1");
   btn[2]  = new Button(100, 325, 100, 75, "2");
   btn[3]  = new Button(200, 325, 100, 75, "3");
   btn[4]  = new Button(0, 250, 100, 75, "4");
   btn[5]  = new Button(100, 250, 100, 75, "5");
   btn[6]  = new Button(200, 250, 100, 75, "6");
   btn[7]  = new Button(0, 175, 100, 75, "7");
   btn[8]  = new Button(100, 175, 100, 75, "8");
   btn[9]  = new Button(200, 175, 100, 75, "9");
   btn[10] = new Button(200, 400, 100, 75, ".");
   btn[11] = new Button(300, 325, 100, 150, "=");
```

Calcクラス

```
    btn[12] = new Button(300, 175, 100, 150, "+");
    btn[13] = new Button(300, 100, 100, 75, "-");
    btn[14] = new Button(200, 100, 100, 75, "*");
    btn[15] = new Button(100, 100, 100, 75, "/");
    btn[16] = new Button(0, 100, 100, 75, "C");
  }

  void display(String str) {
    screen.display(str);
    for (int i=0; i<btn.length; i++) {
      btn[i].display();
    }
  }

  void run() {

    switch(state) {
    case INIT:
      break;

    case INPUT_NUM1:
      break;

    case SELECT_OP:
      break;
```

```
    case INPUT_NUM2:
      break;

    case RESULT:
      break;

    default:
      break;
    }
  }

  String getButtonName(){
    String res = "";
    for(int i=0;i<btn.length;i++){
      if (btn[i].isClicked()){
        res = btn[i].getName();
        break;
      }
    }
    return res;
  }

}
```

未実装

- イベントが起きたかどうかを確認する
  - 数字ボタンを押したかどうか
  - 演算ボタンを押したかどうか
  - ＝ボタンを押したかどうか
  - Cボタンを押したかどうか
- 押したボタンの名前を取得する
- 四則演算
- ほか